

TP 18 — Charger de vraies données

Semaine 18 ■ Fichiers dans NSI/TP18/ ■ 2 h

Objectifs

- Créer, charger et convertir un fichier CSV
- Passer à un vrai jeu de données open data
- Écrire les premières fonctions d'exploitation

Partie A — Premier fichier — `eleves.csv` + `lecture.py`

(30 min)

1. Dans l'éditeur, créer `eleves.csv` (10 lignes de données inventées) :

```
prenom,annee,taille,points
Ada,2010,1.63,15
Blaise,2009,1.71,9
...
```

2. Écrire `charger(nom_fichier)` (cours, avec `csv.DictReader`) et vérifier : `len(table)`, `table[0]`.
3. Écrire `convertir(table)` : année et points en `int`, taille en `float`.
4. Contrôle : `type(table[0]["taille"])` doit afficher `<class 'float'>`.

Partie B — Les premières exploitations — `stats.py`

(40 min)

Sur la table convertie, réinvestir les motifs du cours 11 (les fonctions prennent **la table** en paramètre) :

1. `taille_moyenne(table)`
2. `le_plus_grand(table)` — renvoie le **prénom** de l'élève le plus grand (extremum sur un champ, retour d'un autre!).
3. `nes_avant(table, annee)` — la **liste des prénoms** des élèves nés strictement avant l'année donnée (compréhension).
4. `total_points(table)`
5. ★ `fiche(table, prenom)` — renvoie l'enregistrement du prénom donné, ou `None` (recherche du cours 11, version table).

Partie C — Un vrai jeu de données — `communes.py`

(40 min)

Récupérer un CSV open data (au choix du professeur : communes de France, prénoms de naissance, gares SNCF...). À défaut de réseau, le professeur fournit le fichier. Consignes types pour « communes » (adapter aux colonnes réelles) :

1. Charger, afficher le nombre d'enregistrements et les noms de colonnes (`table[0].keys()`).
2. Convertir la population en entier. Combien de lignes posent problème (champs vides)? Les compter avec un `try` est hors programme : filtrer d'abord les champs non numériques avec `e["population"].isdigit()`.
3. La commune la plus peuplée? La population totale?

4. Combien de communes de moins de 100 habitants ?
5. ★ Les 10 noms de communes les plus longs.

Partie D — Défi ★★ — écrire un CSV

(10 min)

`sauver(table, nom_fichier)` — écrit la table en CSV (sans module : écrire la ligne d'en-têtes puis chaque enregistrement, valeurs jointes par des virgules, avec `f.write(ligne + "\n")` dans un `with open(nom, "w", encoding="utf-8") as f:`). Recharger le fichier produit et vérifier l'aller-retour.